



Wrocław University
of Science and Technology

DISCRETE MATHEMATICS
PROJECT - GROUP 3

Optimization techniques
**Graph-based optimization for resource
allocation during an outbreak**

– Final report –

Students

Szymon CICHY, 235 093
Maxime LUCE, 294 620

Teacher

Marcin BODYCH

Academic year 2025/2026 – 12th June 2026

Contents



Problem Overview and Motivation

During a health crisis such as an epidemic, the rapid distribution of critical resources (e.g., vaccines, medical staff, intensive care beds) is a matter of survival. But the logistics infrastructures (roads, plains, and so on) have limited capacities and they might be issue to close some roads.

This project aims to model this logistical challenge as an optimization problem on a directed network and graph. Our objectives are to design, implement, and analyze algorithms capable of computing the maximum allocation of resources in real time in all scenario possible.

1 General overview on flow networks

1.1 Flow networks and common definitions

In order to explain the *maximum flow* problem, we must introduce the *flow network*.

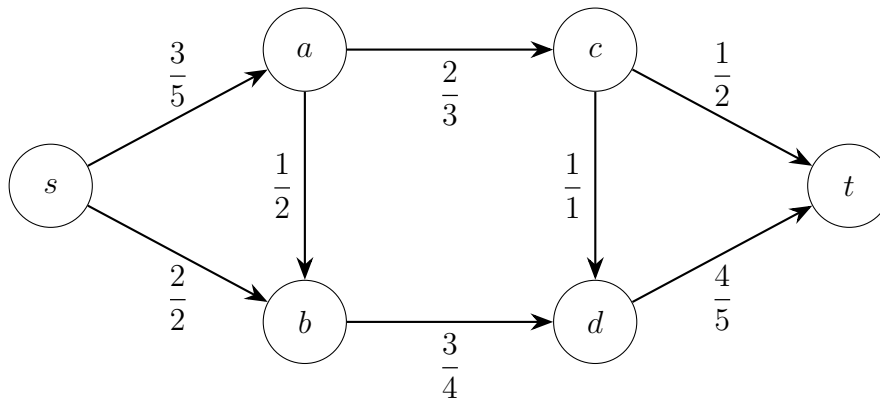


Figure 1: Example flow network

[Flow Network] A directed graph representing transport of flow between nodes. This directed graph can be written as $G = (V, E)$ where V is the set of vertices, $E \subseteq V \times V$ is the set of directed edges, $s \in V$ is the source, $t \in V$ is the sink and $c : E \rightarrow \mathbb{R}_{\geq 0}$ is the capacity function.

[Capacity function] Description

$$\forall (u, v) \in E : \begin{cases} c(v, u) = 0, & \text{if } (v, u) \notin E \end{cases}$$

[Edge Capacity] The maximum amount of flow that an edge can carry.

[Flow] Quantity transmitted through an edge, limited by its *capacity*.

[Node (Vertex)] A point in the network where flow can enter or leave.

[Arc (Edge)] A directed connection between two *nodes*.

[Source] A node with only outgoing flow.

[Sink] A node with only incoming flow.

[Flow function] Description

$$f : E \rightarrow \mathbb{R}_{\geq 0}$$



such that it follows the *capacity constraint* and the *flow conservation*.

[Capacity constraint] Description

$$0 \leq f(u, v) \leq c(u, v) \quad \forall (u, v) \in E$$

[Flow conservation / Kirchoff's Law] Description

$$\sum_{(u,v) \in E} f(u, v) = \sum_{(v,w) \in E} f(v, w) \quad \forall v \in V \setminus \{s, t\}$$

[Excess Function] Excess Function derived from **flow conservation**. It represents the difference between the total incoming and outgoing flow at vertex v . For a flow network $G = (V, E)$ with flow f , the excess function of a vertex v is defined as

$$e(v) = \sum_{(u,v) \in E} f(u, v) - \sum_{(v,w) \in E} f(v, w)$$

- **Conserving** vertex $e(v) = 0$: Flow into the vertex equals flow leaving it.
- **Active** vertex $e(v) > 0$ indicating excess incoming flow. **Sink** t is active
- **Deficient** vertex $e(v) < 0$ indicating more outgoing than incoming flow. **Source** s is deficient

The value of a **feasible flow** (or just **flow**) for a network is the net flow into the sink t :

$$-e(s) = |f| = e(t)$$

1.2 Max flow

$$\max |f|$$

1.2.1 Algorithms

run mostly in polynomial time and give optimal solutions

Algorithm	Year	Time Complexity
Ford–Fulkerson	1956	$O(E \cdot f^*)$
Edmonds–Karp	1969	$O(VE^2)$
Dinic's Algorithm	1970	$O(V^2E)$
Push–Relabel	1986	$O(V^3)$

Table 1: Algorithms for solving the Maximum Flow problem

1.2.2 Maximum flow

is a special case used in other network problem



2 Szymon Part : Flow networks, duality, fairness

2.1 NetworX library and algorithmic approach

We use the `import networkx as nx` python library to define flow network graphs in Python. The flow network is built manually as a `nx.DiGraph` directed graph from an array of tuples.

```

1 G = nx.DiGraph()
2
3 arcs = [
4     ("s", "a", 8),
5     ...
6     ("d", "t", 8),
7 ]
8
9 for u, v, capacity in arcs:
10     G.add_edge(u, v, capacity=capacity)
11 def hello():
12     print("Hello")

```

Without implementing flow algorithms themselves, we can then use various NetworkX methods to solve the basic problem.

```

1 # default using preflow_push()
2 flow_value, flow_dict = nx.max_flow(G, s_source, t_sink)
3 # S, T are partitions after cut
4 cut_value, (S, T) = nx.min_cut(G, s_source, t_sink)
5
6 # we can also use specific algorithms to obtain
7 # a residual flow graph R without building it manually
8 R = nx.algorithms.flow.edmonds_karp(G, s_source, t_sink)
9 flow_dict = nx.algorithms.flow.build_flow_dict(G, R)

```

Finally, we can plot the flow networks using `import matplotlib as plt` library, with the help of networkx own matplotlib-based helpers

```

1 fig, ax = plt.subplots(...)
2
3 nx.draw_networkx_nodes(G, pos, nodelist, ...)
4 nx.draw_networkx_edges(G, pos, edgelist, ...)
5 nx.draw_networkx_labels(...)
6 nx.draw_networkx_edge_labels(...)
7
8 ...
9
10 plt.show()

```



The important insight this approach provides is that the solution for both *max network flow* and *max-flow min-cut* problems is obtained not only by the same algorithm, but a single algorithm pass provides the solution for both by the way of residual graph. When using algorithmic approach, this graph allows us to obtain graph partitions using BFS search, by defining non-reachable nodes as those with no residual capacity

```

1 S = set()
2 q = deque(["s"])
3 S.add("s")
4 while q:
5     u = q.popleft()
6     for v in R.successors(u):
7         residual_capacity = capacity[(u, v)] - flow[(u, v)]
8         if v not in S and residual_capacity > 0:
9             S.add(v)
10            q.append(v)
11
12 min_cut = [(u, v) for u, v in G.edges() if u in S and v not in S]
13 T = set(G.nodes()) - S

```

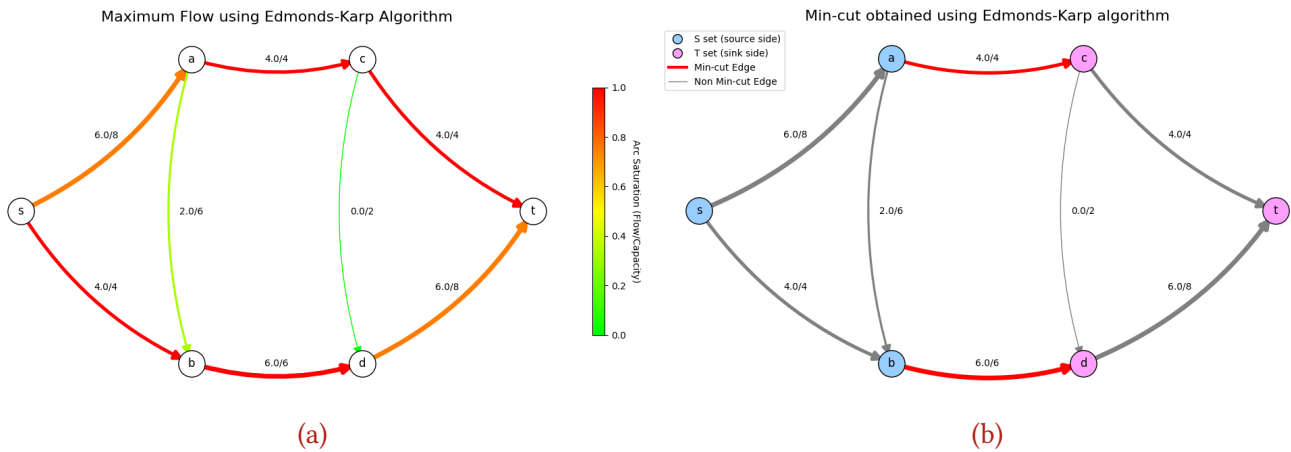


Figure 2: Results of Edmonds-Karp algorithm. The Figure ?? presents edge utility saturation (flow / capacity). Figure ?? highlights node sets and edges in a min-cut obtained using BFS search on a residual flow graph.

2.2 Convex optimization and Linear programming formulation

Linear programming is an optimization method that allows us to obtain a best outcome, defined by a *linear objective function*, constrained by a set of *linear equality and inequality constraints* that form a *feasible region*. This region is geometrically represented by a convex polytope, formed by a set of half-space intersections in the $\mathbb{R}^n$ space (n representing the dimensions of the objective vector). The objective vector x is a vector of *decision variables*, which the linear program calculates. It is a point in the feasible region that represents an optimal solution $f(x^*) = f^* = OPT$. There can be many optimal solutions.

To begin with, we will redefine the max network flow problem in terms of a set of linear equations, using matrix notation.



We start with the following optimization problem:

$$\begin{aligned} & \text{maximize} && f = q^\top x \\ & \text{subject to} && Ax = 0, \\ & && 0 \leq x \leq c. \end{aligned}$$

Let the flow variables be collected in the vector

$$x = \begin{bmatrix} f(e_1) \\ f(e_2) \\ \vdots \\ f(e_m) \end{bmatrix} \in \mathbb{R}^m,$$

where $m = |E|$. Thus, x is the vector of edge flows.

The vector

$$q \in \mathbb{R}^m$$

selects the flow leaving the source s , and is defined by

$$q_i = \begin{cases} 1, & \text{if } e_i = (s, v) \in E, \\ -1, & \text{if } e_i = (v, s) \in E, \\ 0, & \text{otherwise.} \end{cases}$$

Hence, $q^\top x$ equals the net flow leaving the source s . By flow conservation, this is also equal to the net flow entering the sink t .

The capacity vector is

$$c \in \mathbb{R}_{\geq 0}^m,$$

where c_i is the capacity of edge e_i .

Let $n = |V|$. The matrix

$$A \in \mathbb{R}^{(n-2) \times m}$$

is the reduced node-edge incidence matrix, obtained by removing the rows corresponding to the source s and sink t . Its entries are defined by

$$A_{u,i} = \begin{cases} 1 & \text{if edge } e_i = (u, v), \\ -1 & \text{if edge } e_i = (v, u), \\ 0 & \text{otherwise,} \end{cases} \quad u \in V \setminus \{s, t\}.$$

Finally, we can obtain the array representation of the problem for use in cvxpy library

```
1 incidence_mx = np.asarray(nx.incidence_matrix(G, oriented=True).todense())
2 A = incidence_mx[1:-1, :]
3 c = np.array([d['capacity'] for (_, _, d) in G.edges(data=True)])
4 x = cvxpy.Variable(len(G.edges()), nonneg=True)
```



```

5 q = np.array([1 if u == "s" else -1 if v == "s" else 0 for u, v in G.edges])
6
7 # Objective function f = q @ x
8 f = cvxpy.Maximize(q @ x)
9 problem = cvxpy.Problem(f, c)
10 problem.solve()

```

Thus, we have obtained a solution to the max network flow problem. Before showing the results and comparing them to the algorithmic method, let us discuss the other side of this problem - max-flow min-cut. In the previous example, we have obtained a residual graph that contained a solution for both, but because our Linear Programming approach is problem-agnostic: the optimizer does not know that it is solving a graph problem, only that it is finding a vector constrained by a set of equations.

Therefore, in order to solve the max-flow min-cut problem, we have to define another set of variables, constraints and objectives.

$$\begin{aligned}
 & \text{minimize} && d^\top c \\
 & \text{subject to} && d_{uv} - z_u + z_v \geq 0, && \forall (u, v) \in E, u \neq s, v \neq t, \\
 & && d_{sv} + z_v \geq 1, && \forall (s, v) \in E, v \neq t, \\
 & && d_{ut} - z_u \geq 0, && \forall (u, t) \in E, u \neq s, \\
 & && d_{st} \geq 1, && \forall (s, t) \in E, \\
 & \text{sign const.} && d_{uv} \geq 0 \quad \forall (u, v) \in E \\
 & && z_u \in \mathbb{R} \quad \forall (u) \in E \setminus \{s, t\}
 \end{aligned}$$

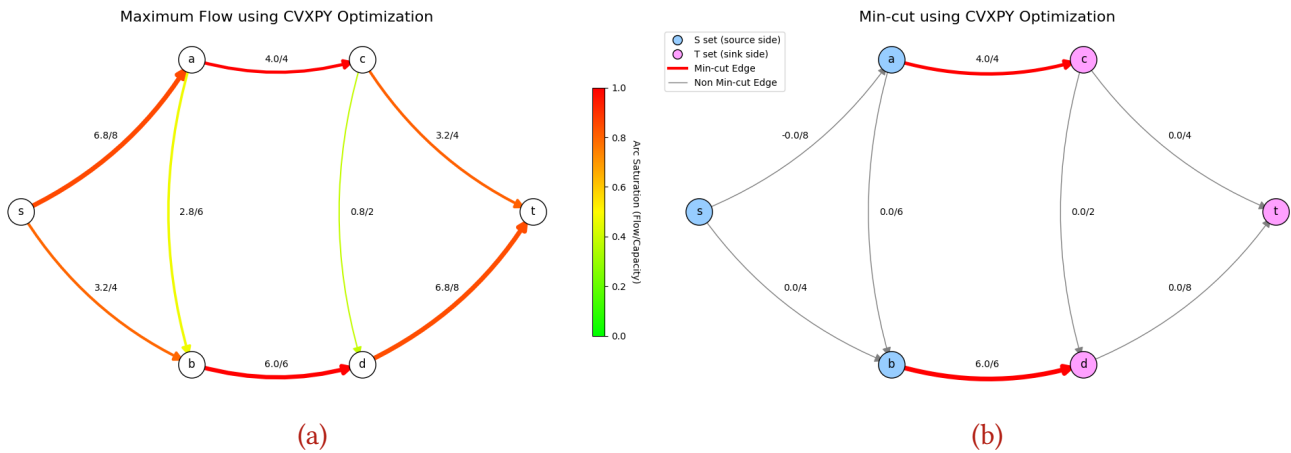


Figure 3: Results of the Linear Program The Figure ?? presents edge utility saturation (flow / capacity). Figure ?? highlights edges included in the cut, obtained by taking d edge variables vector to find the edges and multiplying them by c edge capacity vector to obtain the min-cut value

2.2.1 Algorithmic and LP results comparison

Let us compare algorithmic and linear programming methods. In the case of Edmonds-Karp algorithm, the objective function of maximizing the flow wasn't specified by us - it is defined implicitly by the al-



gorithm itself. The same holds true for the other flow algorithms included in the `networkx` package - maximization of the flow is implicit.

The interpretation of min-cut is also more straightforward - the algorithm terminated when there was no augmenting path from source to sink in the residual graph. The cut edges are the saturated edges crossing between two sets, that we found by using a graph searching algorithm that identified unreachable nodes in the residual graph, and thus, constitute a bottleneck limiting further increase of flow.

In contrast, the linear programming formulation describes the problem as general optimization problem with variables, constraints, and an explicit objective function. Linear programs do not exploit the combinatorial structure of the graph. This necessitated defining constraints for both problems separately, but as we will see, the differences don't end there.

The total flow values returned are identical, as both methods return optimal solution, but the edge flows differ. The "cleaner" appearance of the algorithmic approach is a result of the precise arithmetic nature of the combinatorial approach, while the observed values returned by LP originate from its numerical exploration of the real number space constrained by the linear equations, performed using floating point calculations.

The second difference is the necessity of defining constraints which are implicit in the flow algorithms. In the min-cut solution obtained using `cvxpy`, we can observe that the d and z vector variables contain values only in the $\{0, 1\}$ range, which squares with our interpretation - edge can be either included or not, and the nodes potentials guarantee that edges connecting nodes of different sets are included in the cut. However, if we look at the defined constraints for the min-cut problem, $z_u \in \mathbb{R}, d_{uv} \in \mathbb{R}^+$, allow them to take many real values outside of this range, and solutions which specify the edge inclusion as $d_{uv} = 55.291...$ or $d_{uv} = 0.5$ would be *feasible*, even though conceptually, an edge can't be half-included in a cut. The question is, then, why do optimal solutions push those values towards the limits defined by other constraints.

Finally, we can see that despite the max-flow and min-cut optimization problems defining objective functions and many constraints separately, the resulting value is the same. This happens even though one is a maximization and the other a minimization problem. As we will demonstrate, this is a special example of strong duality - a pair of optimization problems, of which the value of optimal solution to one problem is equal to the other.

2.3 Dual optimization and problem transformation

Every linear optimization problem, which we will call *primal*, can be used to derive a corresponding *dual* problem.

- The variables in the primal correspond to constraints in the dual
- The constraints in the primal become variables in the dual
- The objective directions are reversed - when one is a maximization problem, the other is a minimization problem

The primal problem creates a bound on the dual - for maximization, it is an upper bound. When the bounds are the same on both, then we can say that the *duality gap* is equal to zero. We call this *strong duality*. When there is a gap, it is a case of *weak duality*.

The max-flow min-cut is a case of strong duality. Let us go through the process of deriving a dual in this example.



In general case

$$\begin{aligned} \text{Primal:} \quad & \max \quad c^\top x \\ & \text{s.t.} \quad Ax \leq b, \\ & \quad \quad x \geq 0. \end{aligned}$$

The respective dual is

$$\begin{aligned} \text{Dual:} \quad & \min \quad b^\top y \\ & \text{s.t.} \quad A^\top y \geq c, \\ & \quad \quad y \geq 0. \end{aligned}$$

We can see that the primal constraint b now has a corresponding variable y , and that the variable c became a constraint - now, it is used to define an upper bound that the dual minimizes.

Let us go through the process for max network flow problem

$$\begin{aligned} \text{Max-flow:} \quad & \max \quad q^\top x \\ & \text{s.t.} \quad Ax = 0, \\ & \quad \quad 0 \leq x \leq c. \end{aligned}$$

To derive the dual formulation, we introduce dual variables corresponding to the constraints of the primal max-flow problem. The flow conservation constraint

$$Ax = 0$$

gives rise to the node-potential variables z , while the capacity constraints

$$x \leq c$$

give rise to the edge variables d .

Thus, the primal–dual correspondence is

$$\begin{aligned} Ax = 0 & \quad \longleftrightarrow \quad z, \\ x \leq c & \quad \longleftrightarrow \quad d. \end{aligned}$$

Recall that A is the reduced node-edge incidence matrix, obtained by removing the rows corresponding to the source s and sink t . Consequently, the vector $A^\top z$ contains the potential differences between nonterminal nodes.

The dual problem can therefore be written compactly as

$$\begin{aligned} & \text{minimize} \quad d^\top c \\ & \text{subject to} \quad d - A^\top z \geq q, \\ & \quad \quad d \geq 0. \end{aligned}$$

Here, the vector q encodes the boundary conditions associated with the source and sink. In particular,

$$q_i = \begin{cases} 1, & \text{if } e_i = (s, v) \in E, \\ 0, & \text{otherwise.} \end{cases}$$



Because q already distinguishes edges leaving the source from all other edges, the dual constraints can be expressed uniformly in matrix form, without separately considering the cases $u = s$ or $v = t$. This revelation allowed us to simplify the python code that defines this problem, reducing 12 lines of code into 2.

2.4 Lagrangian and Karush–Kuhn–Tucker conditions

To explain how the primal and dual problems are connected, let us introduce the Lagrangian. The method of Lagrangian multipliers is a way of finding local maxima and minima of a function constrained by a set of equations.

For a standard problem

$$\begin{aligned} \text{Minimize} \quad & f(x) \\ \text{s.t} \quad & g_i(x) \leq 0 \\ & h_j(x) = 0 \end{aligned}$$

The Lagrangian function, or the *Lagrangian*, generalized by the KKT conditions, is defined as

$$\mathcal{L}(x, \lambda, \mu) = f(x) + \sum_i \lambda_i g_i(x) + \sum_j \mu_j h_j(x)$$

Where μ and λ are constants called the Lagrangian multipliers, or in this generalized form, *KKT multipliers*.

In alternative form

$$\alpha = \begin{bmatrix} \mu \\ \lambda \end{bmatrix}, \quad \mu \geq 0,$$

Karush–Kuhn–Tucker theorem

Sufficiency If (x^*, α^*) is a saddle point of the Lagrangian $L(x, \alpha)$ for $x \in X$, $\mu \geq 0$, then x^* is an optimal solution of the optimization problem.

Necessity Suppose that $f(x)$ and $g_i(x)$, $i = 1, \dots, m$, are convex on X , and that there exists $x_0 \in \text{relint}(X)$ such that $g(x_0) < 0$, that is, Slater's condition holds. Then for every optimal solution x^* , there exists a vector

$$\alpha^* = \begin{bmatrix} \mu^* \\ \lambda^* \end{bmatrix}, \quad \mu^* \geq 0,$$

such that (x^*, α^*) is a saddle point of $L(x, \alpha)$.

In plain language, sufficiency states that if a solution to a convex optimization problem, together with a set of multipliers, forms a saddle point of the Lagrangian, then the solution is optimal. In other words, the existence of a saddle point certifies optimality.

Conversely, necessity states that if a solution is optimal, then there exist multipliers such that the corresponding primal-dual pair forms a saddle point of the Lagrangian. Thus, under appropriate regularity conditions, every optimal primal solution has an associated dual solution that certifies its optimality.



Necessary conditions are:

- Stationarity

Minimization:

$$\partial f(x^*) + \sum_i \lambda_i^* \partial g_i(x^*) + \sum_j \mu_j^* \partial h_j(x^*) \ni 0$$

Maximization:

$$\partial f(x^*) - \sum_i \lambda_i^* \partial g_i(x^*) + \sum_j \mu_j^* \partial h_j(x^*) \ni 0$$

- Primal feasibility

$$g_i(x^*) \leq 0, \quad \forall i$$

$$h_j(x^*) = 0, \quad \forall j$$

- Dual feasibility

$$\lambda_i^* \geq 0, \quad \forall i$$

$$\mu_j^* \in \mathbb{R}, \quad \forall j$$

- Complementary slackness

$$\lambda_i^* g_i(x^*) = 0, \quad \forall i$$

Now let us apply the conditions to maximum flow network problem. We have satisfied the *Slater's regular condition* by the way of $x < c$, that is, there exists a solution strictly satisfying all inequality conditions. Both objective function and the linear inequalities are convex.

We will represent the KKT conditions in matrix form.

- **Stationarity** - for the maximization problem, the Lagrangian is

$$L(x, d, z) = q^\top x + z^\top Ax + d^\top (c - x).$$

Rearranging terms by x ,

$$L(x, d, z) = d^\top c + (q + A^\top z - d)^\top x.$$

Taking the gradient with respect to x yields the stationarity condition

$$\nabla_x L(x, d, z) = q + A^\top z - d = 0.$$

Equivalently,

$$d - A^\top z = q.$$



This should remind us of the min-cut inequality constraint,

$$d - A^\top z \geq q.$$

Indeed, this is an important insight into the dual problem. It shows that the edge variable d is not arbitrary, that it has to balance the difference in node potentials z on edges that exist in A . The minimum cut is not just a combinatorial partition of the graph, but also a balancing condition required by the optimality of maximum flow problem.

- **Primal feasibility**

$$Ax = 0, \quad 0 \leq x \leq c.$$

- **Dual feasibility**

$$d \geq 0.$$

This is the source of the sign constraint of the min-cut problem. It guarantees that the edges don't cancel each other out when contributing to min-cut capacity objective, and thus ensuring the dual is truly a minimization problem.

Moreover, this constrains node potentials z by ensuring their values match the edge directions specified in the incidence matrix A .

- **Complementary slackness**

$$d_i(c_i - x_i) = 0, \quad i = 1, \dots, m.$$

Thus, if an edge is not saturated ($x_i < c_i$), then $d_i = 0$; conversely, if $d_i > 0$, the edge capacity constraint is tight $x_i = c_i$. Only fully saturated edges can be contained in the cut.

A final observation is that the vector q introduces a unit source-sink forcing through the constraints $q_{su} = 1$ for edges leaving the source. Through the stationarity condition, this forcing must propagate consistently across the graph via the node potentials

Combined with complementary slackness, the integral, unimodular structure of q and A , this drives the optimal values of d_i toward either 0 or 1. Consequently, an edge either participates in separating the source from the sink, or it does not.

2.5 Introducing fairness

Previously, we have presented the allocation of resources throughout the network as a vector of flows:

$$x = (x_1, x_2, \dots, x_n)$$

In the maximum flow problem, the objective is the maximization of the total resource throughput. We may imagine the capacities as demands or supplies, and the flow as the utilization, or fulfillment of demands. The problem is abstract. In our basic case, the resource is a single, real-valued commodity, and the nodes could be seen as warehouses and hospitals, or as warehouses and regions.

The real resource allocation problem is much more complicated than that. In an epidemic outbreak, the total number of resources delivered, like the number of vaccines, masks or medical supplies distributed,



is rarely the sole objective. In the real world, the problem may be extended to a multi-commodity, multi-objective, dynamic optimization or network planning problem, among others.

In this section, we will examine the concept of fairness as it relates to single real resource distribution - how to allocate it between edges so that no demand is starved, while still being efficient. Pure maximum flow problem may be useful when planning evacuation, where objective is to simply save as many people as possible, but in an epidemic outbreak scenario, if we imagine, for example, edges as hospitals with demand, we want to allocate resources like vaccines more evenly, so they can be distributed to the most vulnerable patients. This is just one scenario.

Pareto Efficiency A solution is *Pareto efficient* if you cannot improve one allocation without hurting another. Formally, allocation x is Pareto efficient if there is no other feasible solution y such that:

$$y_i \geq x_i \quad \forall i$$

and

$$\exists j \in \{1, \dots, n\} \text{ such that } y_j > x_j.$$

Which means every edge is at least as well off, and there is someone strictly better off. If such y exists, then x is not Pareto efficient.

Pareto Frontier is a set of all Pareto efficient solutions. There may be many feasible solutions, and they form a feasible region, however, only the solutions on the "outer edge" of that region offer the best trade-offs.

As we have seen, LP solutions and Edmonds-Karp solutions to maximum flow problem differ, but both are Pareto efficient - flow on one edge cannot be improved without reducing flow on another edge.

The trade-offs between solution could have many definitions, like the flows on e_i vs e_j , or trade-offs between optimization objectives, like total flow versus fairness, however we define it, in our case.

The simple fact that a solution lies on a Pareto frontier does **not** make it fair. By defining an objective function, like the total flow $\max f = \sum_{v \in V} f(s, v)$, we select the trade-off, and that can be interpreted as selecting a single point on a Pareto frontier. As we will see, we can define other objective functions to define different trade-offs.

2.6 Types of fairness

There are multitude of fairness objective definitions, but we will define two useful to us, and then generalize and approximate them in one, single parameter definition.

Min-Max Fairness prioritizes the *worst-off* edges. The goal is simple - improve the smallest allocations until no improvement can be made without decreasing flow of an edge with an equal or smaller allocation.

The standard way of computing min-max fairness is through *progressive filling*. It progressively, iteratively increases utility like *flow/capacity* on edges until they reach they maximum allowable allocation. As it focuses on the worst-off allocations first, and makes it a basis for the second worst and so on, it can be viewed as a *lexicographic* maximization method. We will not be implementing this algorithm, but we can approximate it.



Proportional Fairness Proportional fairness balances efficiency and fairness. Solution x is proportionally fair if for any other feasible y the sum of proportional gain is negative:

$$\sum_i \frac{y_i - x_i}{x_i} \leq 0$$

The interpretation is that improvement for some edges causes larger proportional losses for others. The optimization utility to achieve such outcome is $\log(x)$, and the optimization objective:

$$\max \sum_{(u,v) \in E} \log(x_{uv})$$

For maximum flow objective, the marginal utility (how each unit is valued) is:

$$\frac{d}{dx} x = 1$$

So each added unit is valued equally. For log objective, the utility is valued:

$$\frac{d}{dx} \log(x) = \frac{1}{x}$$

Which is inversely proportional to the current allocation. Increasing flow has diminishing returns and should favor edges with small flows.

Another useful property of logarithms is:

$$\log(ab) = \log a + \log b$$

So maximizing sum of logs is equivalent to maximizing a product:

$$\max \sum_i \log(x_i) \iff \max \prod_i x_i$$

This translates to the optimizer maximizing geometric mean instead of arithmetic mean. Geometric mean should penalize imbalance more.

2.6.1 Alpha Fairness

Alpha fairness is a family of fairness objectives that should unify our definitions:

$$U_\alpha(x) = \begin{cases} \sum_i \frac{x_i^{1-\alpha}}{1-\alpha}, & \alpha \in [0, 1) \cup (1, \infty) \\ \sum_i \log(x_i), & \alpha = 1 \end{cases}$$

Note, that for $\alpha = 0$, the objective is a simple fairness-free total flow.

As for min-max fairness, the solution x^{MMF} can be approximated with high α , as shown in [mo2000fair]:

$$\lim_{\alpha \rightarrow \infty} x^*(\alpha) = x^{\text{MMF}}.$$



This creates a unified definition of our fairness objectives, parametrized by a single parameter α . The parameter α controls how much the objective penalises imbalance: $\alpha = 0$ ignores it entirely, $\alpha = 1$ gives proportional fairness (the Nash bargaining solution), and $\alpha \rightarrow \infty$ maximises only the worst-off allocation. As we will demonstrate, $\alpha = 1$ is often the most appropriate choice under uniform conditions, and increasing α beyond this point can be counterproductive.

2.7 Solving for fairness using convex optimization

We have previously used Linear Programming formulations for max-flow and min-cut problems. In those cases, both objectives and constraints are linear. In order to solve nonlinear objectives such as $\log(x)$, we will use the closely related **convex optimization**. In truth, in our program nothing changes, because the optimizer used was already convex — all linear problems are also convex. This is a generalization of that idea which allows us to define more varied objectives.

Epidemic scenario interpretation. We model an epidemic resource distribution network where the source s is a central depot, inner nodes connected to the source are regional warehouses or hubs, and sink nodes $t_1, t_2, \dots, t_m$ are hospitals or medical centers. Each hospital has a demand D_k representing the resources required. Arcs are transportation routes with capacity $c(e)$ equal to the maximum transport volume per unit time.

The fairness objective operates on the *fraction of demand fulfilled* at each hospital:

$$U_k = \frac{S_k}{D_k}, \quad S_k = \sum_{(u, t_k) \in E} f(u, t_k),$$

where S_k is the total flow arriving at sink k .

The optimisation problem is therefore:

$$\begin{aligned} & \text{maximize} && \sum_{k=1}^m w_k \cdot U_\alpha \left(\frac{S_k}{D_k} \right) \\ & \text{subject to} && 0 \leq f(e) \leq c(e) \quad \forall e \in E, \\ & && \sum_{(u,v) \in E} f(u, v) = \sum_{(v,w) \in E} f(v, w) \quad \forall v \in V \setminus \{s, t_1, \dots, t_m\}. \end{aligned}$$

Here $w_k \geq 0$ are optional per-sink weights. When $w_k = 1$ for all k , the weights are *uniform*.

Code implementation. We use the `cvxpy` library with the `nx.incidence_matrix` approach introduced earlier. The sink-inflow matrix B maps edge flows to per-sink totals:

```
1 incidence_mx = np.asarray(nx.incidence_matrix(G, oriented=True).todense())
2 B = incidence_mx[np.isin(nodes, sinks)]
3 S = B @ flow_vector          # per-sink inflow
4 D = B @ capacity_vector      # per-sink demand
5 U = S / D                    # demand-normalised utility
6 objective = cvxpy.Maximize(cvxpy.sum(cvxpy.log(U + 1e-9)))
```



We sweep $\alpha \in \{0, 0.5, 1, 2, 5\}$ and record per-sink utilisation S_k/D_k and total throughput. Figure ?? shows the results under uniform costs ($w_k = 1$).

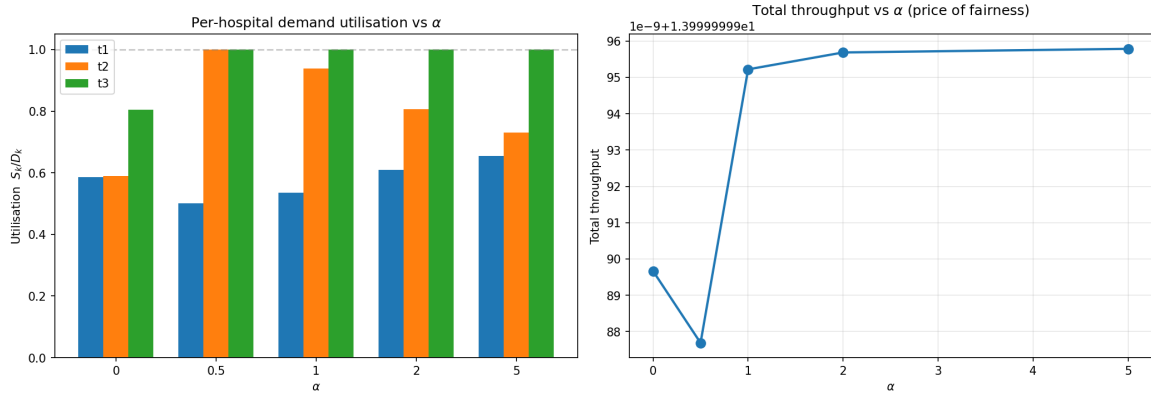


Figure 4: α -fairness spectrum under uniform delivery costs. Left: per-hospital utilisation S_k/D_k across α values. Right: total throughput vs. α . As α increases, utilisation across hospitals becomes more equal but total throughput decreases. At $\alpha = 1$ (proportional fairness), the allocation is already balanced; higher α redistributes flow away from high-demand hospitals without meaningfully helping underserved ones.

Under uniform costs, $\alpha = 1$ achieves proportional fairness — the Nash bargaining solution — where any marginal improvement for one hospital comes at a proportionally larger loss for others. Increasing α beyond 1 further equalises utilisation but reduces total throughput. Crucially, this does not help underserved hospitals: it diverts flow from hospitals with high demand to those with low demand, even though the low-demand hospitals already receive adequate supply at $\alpha = 1$. In the uniform-cost setting, $\alpha = 1$ is the natural sweet spot.

Weighted delivery costs. In practice, hospitals at different distances from the depot face different transportation costs. We introduce per-sink weights w_k to model this asymmetry. Under $\alpha = 1$, the optimiser may favour sinks with higher weights (cheaper to serve), creating cost-driven imbalance.

Figure ?? compares the utilisation profiles under uniform and weighted costs. With weights $w = (1.0, 0.4, 0.2)$ — representing progressively more expensive deliveries — the $\alpha = 1$ solution heavily favours t_1 . Increasing α overcomes this cost pressure by forcing the optimiser to redistribute flow toward underserved (expensive) sinks.

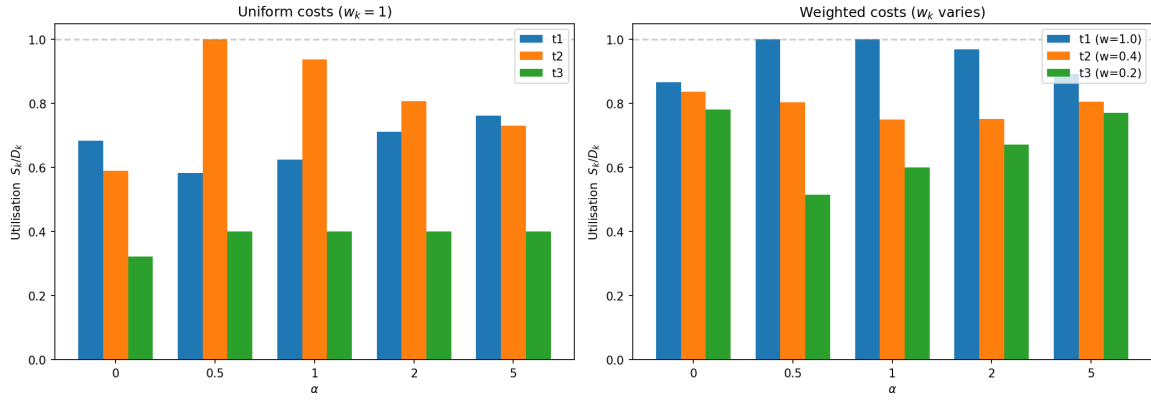


Figure 5: Fairness under cost asymmetry. Left: uniform costs ($w_k = 1$) – $\alpha = 1$ already balances the allocation. Right: weighted costs ($w = 1.0, 0.4, 0.2$) – $\alpha = 1$ favours the cheapest sink; increasing α redistributes flow toward expensive, underserved hospitals.

The throughput comparison in Figure ?? shows that the *cost of fairness* – the throughput lost to achieve egalitarianism – is higher under weighted costs. This is the price of overcoming cost-driven bias.



Figure 6: Cost of fairness: uniform vs. weighted delivery costs. Under uniform costs, throughput decreases smoothly with α . Under weighted costs, a larger throughput sacrifice is needed to overcome cost-driven bias toward cheaper sinks.

2.8 Cost of fairness – Pareto frontier and dual interpretation

The *Price of Fairness* (PoF) quantifies the throughput sacrificed in pursuit of fairness:

$$\text{PoF} = \frac{f_{\max} - f_{\text{fair}}}{f_{\max}},$$

where $f_{\max}$ is the maximum total flow (utilitarian, $\alpha = 0$) and f_{fair} is the throughput under a fairness objective.

We trace the **Pareto frontier** between throughput and Jain's fairness index

$$J = \frac{(\sum_k U_k)^2}{m \sum_k U_k^2}$$

by solving a sequence of constrained problems: (a) fix a target throughput and maximise fairness, and (b) fix a target Jain index and maximise throughput. Each point on the frontier represents a different efficiency-equity trade-off.



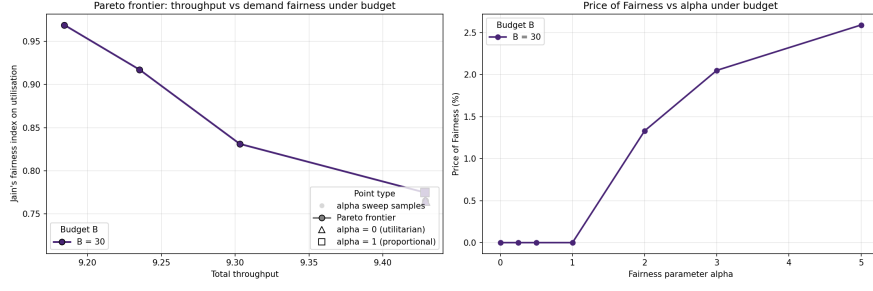


Figure 7: Pareto frontier: efficiency vs. fairness. The frontier traces all achievable (throughput, Jain index) pairs. The utilitarian optimum ($\alpha = 0$) achieves maximum throughput but lowest fairness; the egalitarian solution achieves maximum fairness at the cost of throughput. The Price of Fairness is the horizontal gap between these extremes.

Lagrangian dual and shadow prices. For the log-utility case ($\alpha = 1$), the Lagrangian of the fairness problem is

$$\mathcal{L}(f, \lambda, \mu) = \sum_k \log\left(\frac{S_k}{D_k}\right) + \mu^\top A_{\text{cons}} f + \lambda^\top (c - f),$$

where $\lambda_e \geq 0$ are the dual variables for the capacity constraints and μ_u are the node potentials from flow conservation. The stationarity condition gives:

$$\frac{\partial \mathcal{L}}{\partial f_e} = 0 \implies \sum_{k: e \text{ reaches } t_k} \frac{D_k}{S_k} + (A_{\text{cons}}^\top \mu)_e - \lambda_e = 0.$$

This reveals the economic interpretation:

- λ_e acts as a *congestion toll* on edge e . By complementary slackness, $\lambda_e > 0$ only when the edge is saturated ($f_e = c_e$).
- μ_u acts as a *node potential* measuring the marginal value of one additional unit of flow at node u .
- The term D_k/S_k is the marginal utility of flow to hospital k – it is high when the hospital is under-served (low S_k relative to D_k) and low when well-served.

Complementary slackness $\lambda_e(c_e - f_e) = 0$ guarantees that only fully saturated edges carry a positive congestion toll. This connects the fairness dual to the max-flow dual discussed earlier: while the max-flow dual has integral $\{0, 1\}$ solutions (due to total unimodularity), the fairness dual produces continuous shadow prices that reflect the *marginal value of capacity* in terms of utility rather than raw flow.

3 Maxime Part : Study of sensitivity

3.1 Context

Based on the second presentation on the topic "Shortest path algorithms in weighted graphs", we had use and improve the graph that represent Wroclaw city. The Figure ?? presents the final graph that we will use to run our experiments.



In this graph, one can see the major position of the city such as the *lotnisko*, fire station, the water plant or the hospital.

Regarding the capacity scale, we apply the following rules for the tree type of category:

- weight of 50 and more when it involves highwyas or major arteries such as the A4 or Legnicka ;
- weight between 20 and 40 for standard city avenues such as the *Plac Grunwaldzki* ;
- weight between 5 and 15 for the small streets, historic bridges and residential areas.



img/wroclaw_graph.png

Figure 8: Wroclaw graph uses as a context for our work.

3.2 Sensitivity to capacity augmentation: duality and minimum cuts

Our goal is to *identify the edges where increasing capacity improves overall throughput*, and those where it has no impact.

First, we will present the mathematical background to understand and solve this problem ??, then we will present a naive method to find those edges ?. Finally, we will present a optimized method based on a powerful mathematical result ?.

3.2.1 Theoretical review

The Max-Flow problem seeks to send the maximum amount of resources from a source s to a sink t in a directed graph $G = (V, E)$, where each edge (i, j) has a maximum capacity c_{ij} .

Mathematically, we maximize the total flow v subject to the Kirchhoff's law (??) and the capacity constraint (??).

The Kirchhoff's law states that for any node $k \in V \setminus \{s, t\}$, the sum of incoming flows equals the sum of outgoing flows:

$$\sum_i f_{ik} - \sum_j f_{kj} = 0 \quad (1)$$

The capacity constraint states that we cannot exceed the capacity of each edge.

$$\forall (i, j) \in E, \quad 0 \leq f_{ij} \leq c_{ij} \quad (2)$$

The fundamental theorem states that the maximum flow is exactly equal to the capacity of the minimum cut (min-cut). A cut (S, T) is a partition of the nodes such that $s \in S$ and $t \in T$. Its capacity is the sum of the capacities of the edges directed from S to T .

If you increase the capacity c_{ij} of an edge by a value δ , the total flow v will only increase if the edge (i, j) belongs to the limiting minimum cut.

In terms of Linear Programming, sensitivity analysis corresponds to dual variables (or shadow prices). The dual variable associated with the capacity constraint $f_{ij} \leq c_{ij}$ indicates the marginal gain on the objective function (the flow) if this constraint is relaxed by one unit.

On the one hand, if the shadow is strictly positive, the edge is a bottleneck (in the minimum cut). Increasing c_{ij} increases the total flow. On the other hand, if the shadow price is zero, the edge's capacity is not saturated, or saturating it does not limit the overall flow. Increasing c_{ij} is a waste of epidemiological resources.

3.2.2 Naive method

First, the naive method can be considered: given a change δ on the edges capacity, we can for instance compute the total flow for every single edge of the graph and see if it improves the maximum flow.

[caption=Python code for the naive approach, label=lst:1naive_method]def naive_sensitivity(G, s, t, delta = 1) : """ Naive approach : Test every single edge to see if increasing its capacity improves the maximum flow. """
base_flow_value = nx.maximum_flow_value(G, s, t)
sensitivity_results =

for u, v, data in G.edges(data=True): original_capacity = data['capacity']
G[u][v]['capacity'] = original_capacity + delta

new_flow_value = nx.maximum_flow_value(G, s, t)
sensitivity_results[(u, v)] = new_flow_value - base_flow_value

G[u][v]['capacity'] = original_capacity

return sensitivity_results



3.2.3 Optimized method

However, the naive method is not optimal as we need to test every single edge. That is the reason why we can apply the *Max-Flow Min-Cut Theorem* that states that if you increase the capacity c_{ij} of and edge by a certain value δ , the total flow will only increase if the edge belongs to the limiting minimum cut.

Therefore, we do not need to test every single edge, we only need to find and test the edges that are in limiting minimum cut. Hence, we are able to improve our method into an optimized method (*1_optimized_method*).

```
[caption=Python code for the naive approach, label=lst:1_optimized_method]def optimized_sensitivity(G, s, t, delta):
1) : """ Optimized approach : Only tested edges that cross the minimums - t cut. """
base_flow_value = nx.maximum_flow_value(G, s, t)
sensitivity_results = (u, v) : 0 for u, v in G.edges()
```

```
Find the minimum cut partition cut_value, partition = nx.minimum_cut(G, s, t)
reachable, non_reachable = partition
```

```
cut_edges = [(u, v) for u, v in G.edges() if u in reachable and v in non_reachable]
```

```
for u, v in cut_edges : original_capacity = G[u][v]['capacity']
G[u][v]['capacity'] = original_capacity + delta
```

```
new_flow_value = nx.maximum_flow_value(G, s, t)
sensitivity_results[(u, v)] = new_flow_value - base_flow_value
```

```
G[u][v]['capacity'] = original_capacity
```

```
return sensitivity_results
```

3.2.4 Comparison and analysis

As the naive method test every single edge of the graph, by noting the total number of edges $|E|$ (cardinal), this method requires exactly $1 + |E|$ number of iterations. However, the optimized method test all edges within the min-cut (Table ??).

Method	Number of iterations
Naive	$1 + E $
Optimized	$1 + E_{\text{cut}} $

Table 2: Comparison of total number of iterations for each

In order to compare these two methods, we will run a benchmark. First, we will generate a random Erdős-Rényi oriented graph and we will set a random capacity between 1 and 15. We make sure to check if there is a path from the source to the sink because, otherwise, the total flow is null and the test is not relevant.

Then we can track the evolution of computation time in function of the size of the random generated graph (Figure ??). On the normal scale, we can see how efficient the optimized method is comparing to the naive method. As the graph's size are all in the same order, the cardinal of the min-cut is similar which explains the low-growing function that we observe.



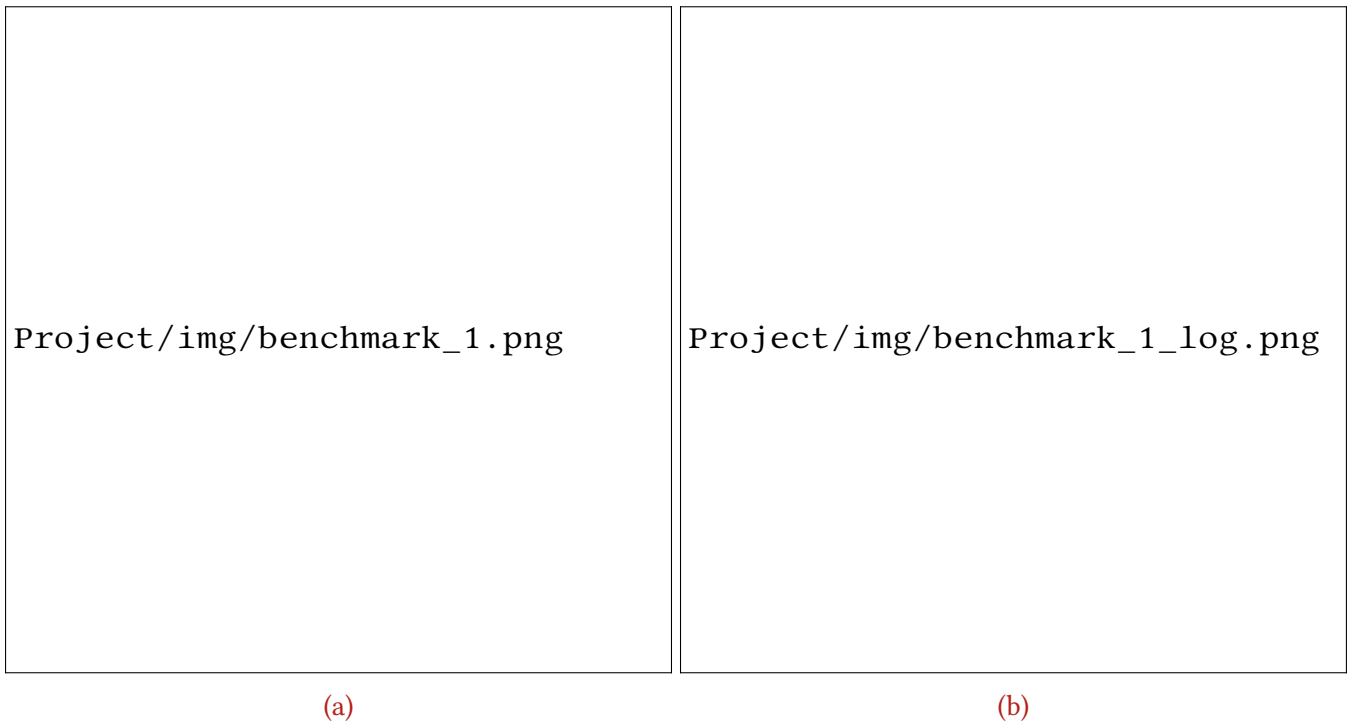


Figure 9: Comparison between the naive and the optimized method. The **Figure ??** present the performance with the number of nodes in the graph and the **Figure ??** present the same evolution using a log scale.

This execution time comparison illustrates the computational overhead of recalculating flows and cuts, which further justifies avoiding iterative recalculation on large graphs, paving the way for the MILP formulation.

3.2.5 Main observations

Let's consider the graph **Figure ??**. We can see that the initial max flow is 11. We observe that, if we add +10 to a non-bottleneck edge (between S and A), then the new flow is also 11. Therefore, adding capacity to a non-bottleneck edge is absolutely irrelevant.

Then, if we add +10 to a true bottleneck between node A and T . Then, the new flow is 15 so we have a gain of +4. What is the reason for that? In fact, the edge between node S and A became the *new* bottleneck.

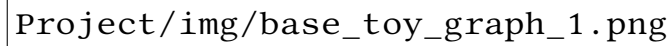
Project/img/base_toy_graph_1.png

Figure 10: Simple graph used to illustrate the case of a graph with multiple bottlenecks.

Let's now consider the case where we have multiple min-cut edge. In order to study the impact on the total flow, let's consider the graph illustrated in **Figure ??**.

The initial max flow is 10 and we will upgrade on min cut (the edge between nodes S and A) by $+5$. We observe then that the new flow is the same, the flow didn't increase because we are still constrained by the edge between A and T .

We must therefore conclude that, in order to increase the flow, we need to upgrade systematically all simultaneous cuts.

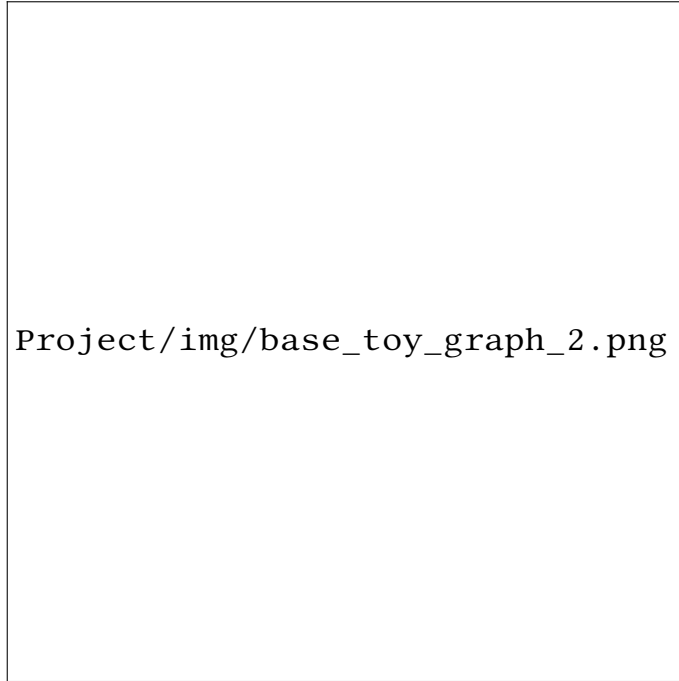


Figure 11: Simple graph used to illustrate the need to upgrade all simultaneous cuts in order to increase the total flow.

As demonstrated above, upgrading a single edge changes the entire bottleneck structure of the network. Because the number of possible cuts grows exponentially ($O(2^n)$), calculating all min-cuts to decide on multiple upgrades is highly inefficient. Furthermore, a Greedy approach fails here because an irrevocable upgrade choice in one iteration might lead to sub-optimal bottlenecks in the next. This proves that assigning a multi-edge budget is a complex Combinatorial / NP-hard problem.

3.3 Identification of the most critical edge

Our goal is now to measure the flow loss if a ridge is completely removed (or if its capacity is reduced to zero). This can be the case if, for instance, a road is removed or if an unexpected issue occurs.

First, we will present the mathematical background to understand and solve this problem ??, then we will present a naive method to find those edges ?. Finally, we will present a optimized method based on a powerful mathematical result ?.

3.3.1 Theoretical review

Here, we are no longer looking at expansion, but at the resilience of the network (e.g., what happens if a route is cut off by floods or if a warehouse fails?).

We define the "vitality" of an edge e by the decrease in the maximum flow value $\Delta v(e)$ when this edge is removed from the network:

$$\Delta v(e) = |f|_{\max}(G) - |f|_{\max}(G \setminus \{e\}) \quad (3)$$

We will explore the mathematical properties of this quantity.

First, as the the flow loss cannot exceed the capacity of the removed edge, nor the actual flow passing through it, we can provide an upper bound (??).

$$\Delta v(e) \leq f(e) \leq c(e) \quad (4)$$

Then, we can discuss the connection of the vitality with the minimum cut. An edge e has a vitality $\Delta v(e) > 0$ if and only if it belongs to all possible minimum cuts of the graph. If it only belongs to some minimum cuts, the flow can find another path that will saturate an alternative cut, and the loss will be strictly less than $c(e)$.

The comparison is indeed why the identification of the minimum cut edges is an excellent pre-selection heuristic to significantly reduce computational time (TO DEVELOPP).

3.3.2 Naive method

First, the naive method can be considered: for every single edge in the graph, we can simulate his removal, compute the final total flow and save it. At the end, we return the edge that causes the maximum flow.

[caption=Python code for the naive approach in order to identify the most vital edge, label=lst:2naive_methodlst]
 """Identifiesthe'mostvitaledge'inaflownetwork.Themostvitaledgeistheonewhoseremovalcausesthelargest
 nx.maximum_flow_value(G, s, t)

```
most_vital_edge = Nonemax_flow_loss = -1
edge_losses =
for u, v, data in G.edges(data=True): original_capacity = data['capacity'] if original_capacity == 0 :
continue
Simulate edge removal G[u][v]['capacity'] = 0 new_flow_value = nx.maximum_flow_value(G, s, t)
flow_loss = base_flow_value - new_flow_value edge_losses[(u, v)] = flow_loss
if flow_loss > max_flow_loss : max_flow_loss = flow_loss most_vital_edge = (u, v)
G[u][v]['capacity'] = original_capacity
return most_vital_edge, max_flow_loss, edge_losses
```

3.3.3 Optimized method

In fact, if the edge has no flow, his loss will be 0. We can therefore save all these edges and avoid useless computation.

[caption=Python code for the naive approach in order to identify the most vital edge, label=lst:2naive_methodlst]
 computetheflowandrecuperationoftheflowdistributionbase_flow_value, flow_dict = nx.maximum_flow(G, s, t)

```
most_vital_edge = Nonemax_flow_loss = -1
edge_losses = dicttosaveallthelosses
for u, v, data in G.edges(data=True): original_capacity = data['capacity']
ignore the edges that haven't any capacity if original_capacity == 0 : edge_losses[(u, v)] = 0 continue
optimization here if the edge hasn't flow, his loss is 0 we save it to the dict and we do not compute
again the total flow if flow_dict[u].get(v, 0) == 0 : edge_losses[(u, v)] = 0 continue
simulation of the releting only for active edges G[u][v]['capacity'] = 0 new_flow_value = nx.maximum_flow_value(G, s, t)
flow_loss = base_flow_value - new_flow_value edge_losses[(u, v)] = flow_loss
if flow_loss > max_flow_loss : max_flow_loss = flow_loss most_vital_edge = (u, v)
```



```

restoration of the original capacity  $G[u][v]['capacity'] = original\_capacity$ 
return  $most\_ital\_edge, max\_flow\_loss, edge\_losses$ 

```

3.3.4 Comparison and analysis

In order to compare these two methods, we will run the same benchmark. Then we can track the evolution of computation time in function of the size of the random generated graph (Figure ??). On the normal scale, we can see how efficient the optimized method is comparing to the naive method. As the graph's size are all in the same order, the cardinal of the min-cut is similar which explains the low-growing function that we observe.

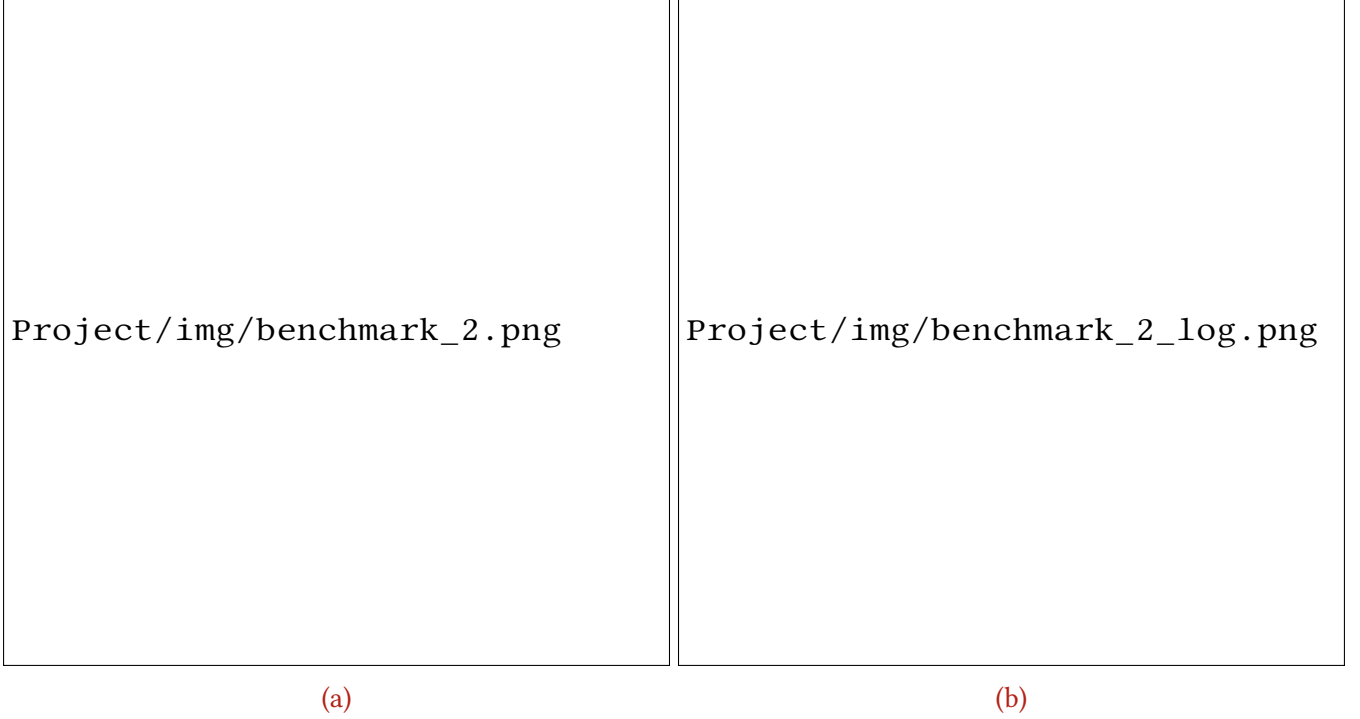


Figure 12: Comparison between the naive and the optimized method. The Figure ?? present the performance with the number of nodes in the graph and the Figure ?? present the same evolution using a log scale.

3.4 Optimal sequence of k edges subject to a budget constraint

In section ??, we established that a greedy edge-by-edge upgrade strategy is mathematically flawed for holistic network planning. Therefore, to find the true optimal combination of upgrades within a set budget, we must use Mixed-Integer Linear Programming (MILP) which evaluates the entire search tree globally rather than myopically.

3.4.1 Theoretical review

We transition from a simple analysis problem to a Network Design Problem, where we have a certain budget B . Increasing the capacity of an edge (i, j) by one unit costs w_{ij} . We introduce a new decision variable $y_{ij} \geq 0$ representing the added capacity.



The new linear program becomes therefore to maximize v under the following constraints:

$$\begin{cases} \forall k \in V \setminus \{s, t\}, \quad \sum_i f_{ik} - \sum_j f_{kj} = 0 & \text{(Kirchoff's law)} \\ \forall (i, j) \in E, \quad 0 \leq f_{ij} \leq c_{ij} + y_{ij} & \text{(Capacity's constraint)} \\ \sum_{(i,j) \in E} w_{ij} y_{ij} \leq B & \text{(Budget's constraint)} \end{cases} \quad (5)$$

3.4.2 Greedy algorithm

The greedy method aims to iteratively select the best single edge to upgrade (3_{greedy_method}).

```
[caption=Python code for the greedy approach in order to identify the sequence of  $k$  edges under bud-
get constraint, label=lst:3greedy_method]def greedy_k_edges(G, s, t, k, delta = 1.0) : """Greedy approach :
Iteratively select the best single edge to upgrade.""" G_temp = G.copy() chosen_edges = []

for i in range(k): best_edge, max_gain = None, 0 base_flow = nx.maximum_flow_value(G_temp, s, t)

for u, v in G_temp.edges() : G_temp[u][v]['capacity'] += delta gain = nx.maximum_flow_value(G_temp, s, t) -
base_flow

if gain > max_gain : max_gain = gain best_edge = (u, v)

G_temp[u][v]['capacity'] -= delta

if best_edge : chosen_edges.append(best_edge) G_temp[best_edge[0]][best_edge[1]]['capacity'] += delta else :
break

return chosen_edges, nx.maximum_flow_value(G_temp, s, t)
```

However, the greedy algorithm is not always optimal because the maximum flow function is not strictly submodular with respect to capacity addition.

In discrete mathematics, a greedy algorithm offers strong approximation guarantees if the objective function exhibits diminishing marginal returns (submodularity). However, in a network, increasing the capacity of route A might yield zero gain until you also increase the capacity of the subsequent route B (synergy effect). The greedy approach will ignore A and B because their immediate individual gain is 0, whereas the joint investment in $(A + B)$ would have been the global optimum.

3.4.3 Linear Programming

The LP Solver (implemented in pulp module), by evaluating the solution space holistically (using the simplex algorithm or interior point methods), will capture this synergy that the greedy algorithm misses (3_{lp_method}).

```
[caption=Python code for the greedy approach in order to identify the sequence of  $k$  edges under bud-
get constraint, label=lst:3lp_method]def exact_k_edges_lp(G, s, t, k, delta = 1.0) : """Exact approach using Mixed
integer programming using pulp.LpProblem("Max_Flow_Budget", pulp.LpMaximize) flows, upgrades = ,

for u, v in G.edges(): flows[(u, v)] = pulp.LpVariable(f"flow_{uv}", lowBound = 0) upgrades[(u, v)] =
pulp.LpVariable(f"up_{uv}", cat = pulp.LpBinary)

prob += pulp.lpSum([flows[(s, v)] for v in G.successors(s)]), "Total_Flow" prob += pulp.lpSum(upgrades.values()),
k, "Budget"
```



```

for u, v, data in G.edges(data=True): basecap = data['capacity']prob += flows[(u, v)] <= basecap +
(upgrades[(u, v)] * delta)
for node in G.nodes(): if node not in [s, t]: prob += pulp.lpSum([flows[(u, node)] for u in G.predecessors(node)])
== pulp.lpSum([flows[(node, v)] for v in G.successors(node)])
prob.solve(pulp.PULP_CBC_CMD(msg = False))
chosenedges = [(u, v) for u, v in G.edges() if pulp.value(upgrades[(u, v)]) > 0.5] return chosenedges, pulp.value

```

3.4.4 Comparison and analysis

Finally, we will continue to use generated random graph to compare both methods. On the one hand, we can follow the evolution of the computational time with the size of the graph (Figure ??). On the other compare, we can study how the greedy method is in fact close to the optimal solution (Figure ??).

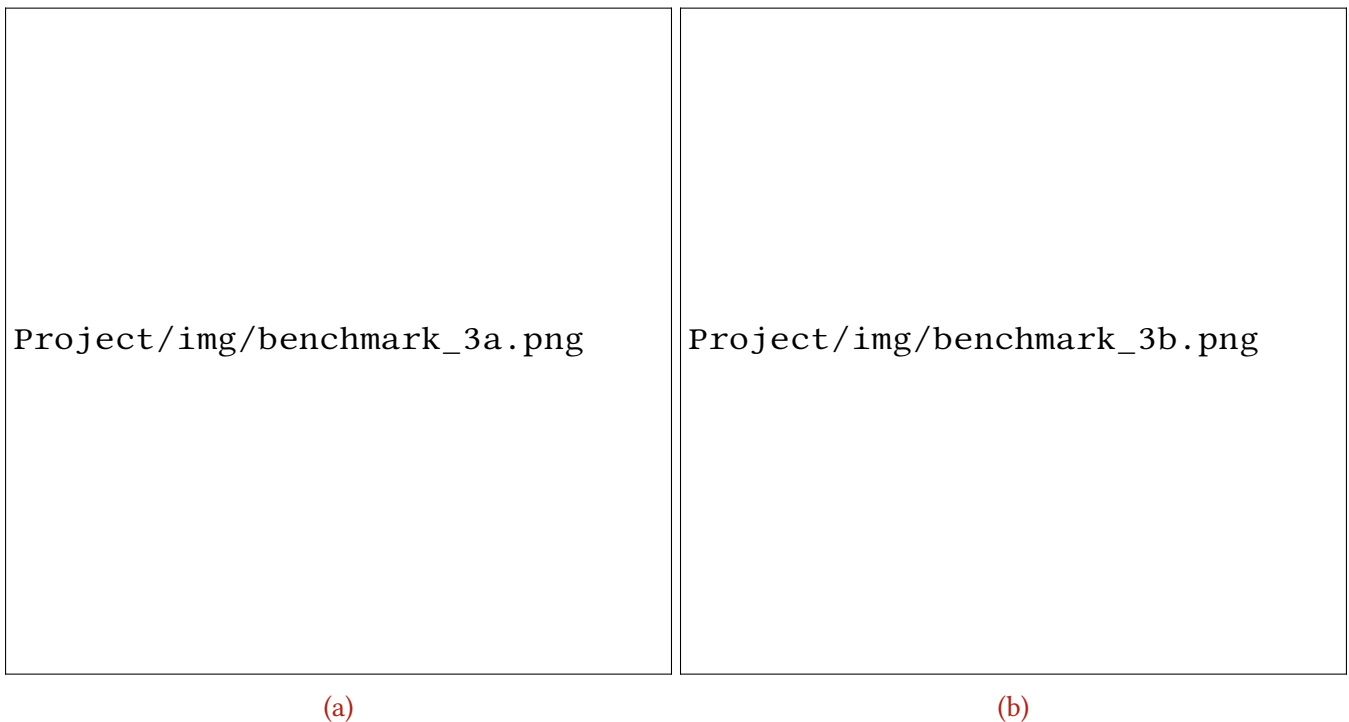


Figure 13: Comparison between the greedy and the linear programming. The Figure ?? present the execution time with the number of nodes in the graph and the Figure ?? present efficiency of the greedy algorithm comparing with the real optimal solution.

4 Further development (as homework ?)

Conclusion

References



Appendixes

List of Tables

List of Figures

Listings

